

Which Fix Should We Test? Budgeted Causal Verification of LLM Failure Attributions for Self-Improving Agents

Anonymous ACL submission

Abstract

Self-improving LLM agents increasingly rely on an LLM to read failed trajectories, name a root cause, and decide which part of the agent to change. These attributions are cheap but unreliable, and checking them by replaying the agent with a candidate fix is reliable but expensive. We ask which attributions are worth checking under a fixed replay budget. We cast the problem as learning a causal bipartite graph from failure categories to modifiable agent components, where each edge is an unknown intervention effect. Our method, CARVE, places a Bayesian model on this graph that separates whether a component is a cause from how good a particular patch is, uses LLM attributions as a calibrated prior rather than as a verdict, and accounts for noisy category labels. It chooses the next intervention (which category, which component, which patch, and whether to run a cheap single-step check or a full replay) by maximising the expected decision value gained per unit cost. Two short propositions characterise when the LLM prior saves replays and when cheap checks pay off, and both predictions match our simulations. In a controlled testbed with known ground truth, CARVE recovers 79% of the oracle’s repairable failure mass at a budget of 50 full replays, against 58% for uncertainty sampling and 27% for a do-then-verify baseline; inside a multi-round self-improvement loop it reaches in two rounds a success rate that the strongest baseline reaches only after six. We release the method and a fault-injected tool-use environment for real-agent studies.

1 Introduction

A growing family of systems improves LLM agents from their own failures: the agent runs, an analyst model reads the failed trajectories, and its diagnosis drives a change to a prompt, a tool, or the orchestration code (Shinn et al., 2023; Zhao et al., 2024; Hu et al., 2025; Zhang et al., 2025a; Agrawal

et al., 2025). The diagnosis step has become a research topic in its own right under the name *failure attribution* (Zhang et al., 2025b, 2026; Zhu et al., 2025). Two findings from that literature motivate this paper.

First, LLM attribution is unreliable. On the recent Who&When Pro benchmark the strongest model reaches 74% step-level accuracy on text traces but only 22% F1 on identifying the error mode, and accuracy falls to roughly 50% on traces longer than 12k tokens (Liu et al., 2026). Models tend to label failures by their visible symptom rather than their cause. A self-improvement loop that trusts these labels will often change the wrong component, and a wrong change can hurt: accurate failure prediction does not imply that acting on it helps (Vasudev et al., 2026).

Second, attributions can be checked by intervention. If a diagnosis says that a tool description caused a failure, one can patch that description, replay the trajectory from the failing step, and see whether the task now succeeds (Ma et al., 2025; Lin et al., 2026; Bonagiri et al., 2026; Shah, 2026). This is the agent analogue of a *do*(·) experiment (Pearl, 2009), and it is far more reliable than reading the trace. It is also expensive: each check costs a partial agent rollout, outcomes are stochastic so one replay is rarely conclusive, and a failed check is ambiguous because the patch itself may simply be bad.

Existing intervention-based methods verify one hypothesis at a time, usually with a fixed number of replays per hypothesis. In a self-improvement loop the question is different. There are many failure categories and many components that could be changed, the replay budget is fixed, and what matters is not the accuracy of any single diagnosis but how much of the agent’s failure mass is repaired at the end. We therefore ask: *given a budget of interventions, which attributions should be verified, with which patch, and how cheaply?*

We make the following contributions.

- We formulate budgeted verification as learning a *category–component causal graph* whose edges are unknown intervention effects, and we separate the causal question (is component j a cause of category i ?) from the engineering question (is this particular patch any good?) by giving each cell several candidate patches (§3).
- We propose CARVE (Cost-Aware Replay for Verifying Edges), a Bayesian method that uses LLM attributions as a calibrated prior, models label noise and spurious recovery, and selects interventions across cells, patches and two fidelities (a cheap single-step check and a full replay) with a multi-step knowledge-gradient rule divided by cost (§4).
- We give two propositions that say when the LLM prior saves replays and when a cheap check is worth running, in terms of quantities a practitioner can estimate (§5).
- In a controlled testbed with known ground truth, CARVE needs roughly a third of the budget of the strongest baseline to reach the same repaired failure mass, never does worse than a flat prior even when LLM attributions are nearly useless, and converts that efficiency into faster agent improvement inside a multi-round loop (§6). We also release ShopDesk, a deterministic tool-use environment with injected faults that provides ground truth for real-LLM studies (§7).

2 Related Work

Failure attribution for LLM agents. Zhang et al. (2025b) introduced the Who&When benchmark and compared all-at-once, step-by-step and binary-search prompting for locating the responsible agent and step; Liu et al. (2026) extended it to longer and multimodal traces and found that attribution remains far from solved. AgenTracer uses counterfactual replay and fault injection to build training data for a small attribution model (Zhang et al., 2026); AgentDebug pairs an error taxonomy with targeted corrective feedback (Zhu et al., 2025); ErrorProbe traces dependencies backwards and keeps a verified memory of error patterns (Li et al., 2026). Taxonomies such as MAST (Cemri et al., 2025) give attribution a fixed vocabulary,

and AdaMAST induces a system-specific taxonomy from traces whose axes map to intervention points (Cemri et al., 2026). AdaMAST validates its taxonomy by agreement between LLM annotators; we instead treat the mapping from categories to components as a causal claim to be checked by intervention.

Verification by intervention. DoVer generates hypotheses from logs and validates each by editing a message or plan and re-running, with several runs per intervention (Ma et al., 2025). REFLECT tests diagnosis-specific patches by targeted replay and feeds verified outcome changes back into attribution (Lin et al., 2026). CausalFlow scores steps by their causal responsibility under counterfactual interventions (Bonagiri et al., 2026), and Causal Agent Replay formalises step interventions in a structural causal model with a point-of-commitment rule (Shah, 2026). AgentDebugX provides a toolkit with policy-gated reruns and a store of failure–diagnosis–repair bundles (Zhu et al., 2026). All of these operate per trajectory or per hypothesis and do not decide how to allocate a shared budget across hypotheses, patches and levels of fidelity, which is our focus. Our method can use any of them as the replay primitive.

Self-improving agents. Reflexion, Self-Refine and ExpeL turn feedback into verbal lessons (Shinn et al., 2023; Madaan et al., 2023; Zhao et al., 2024); ADAS, the Darwin Gödel Machine and GEPA search over agent designs or prompts guided by evaluation and failure logs (Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2025). HarnessFix attributes failures to harness artefacts and repairs them (Chen et al., 2026). Bayesian-Agent keeps Beta posteriors over the reliability of skills and applies threshold rules to patch, split or retire them (Wu et al., 2026); its posterior is over skills rather than causal edges, and actions are chosen by fixed thresholds rather than by the value of information. CARVE is complementary: it decides which proposed fixes to trust before they are applied.

Experimental design. Our selection rule is a knowledge-gradient policy (Frazier et al., 2008) within Bayesian experimental design (Chaloner and Verdinelli, 1995), with multi-fidelity observations (Kandasamy et al., 2016) and a thresholding objective (Locatelli et al., 2016); the stopping behaviour mirrors sequential testing (Wald, 1945). Active learning of causal structure from interventions is

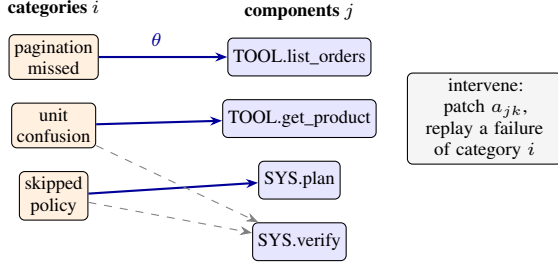


Figure 1: The category–component graph. Solid edges are true causal edges; dashed ones are plausible but wrong attributions (e.g. blaming verification for a unit error). Each intervention patches one component and replays one failure.

classical (Hauser and Bühlmann, 2014). The contribution here is not a new design criterion but a formulation of agent self-improvement in which these tools apply, together with a model of the specific nuisances (bad patches, noisy labels, spurious recovery, LLM priors) that arise there.

3 Problem Formulation

An agent is built from *components* $j \in \{1, \dots, J\}$: sections of its system prompt, tool descriptions, tool implementations, orchestration settings. Its failures have been grouped into *categories* $i \in \{1, \dots, I\}$ by an LLM, with f_i the share of failures in category i . For every cell (i, j) an analyst LLM can propose K candidate patches a_{ijk} that rewrite component j to address category i .

Interventions. An intervention (i, j, k, ℓ) samples a failed trajectory labelled i , applies patch a_{ijk} , and observes a binary outcome at fidelity ℓ . A *full replay* ($\ell = F$) resumes the episode from the attributed failing step and records whether the task now succeeds. A *single-step check* ($\ell = S$) regenerates only the failing step and asks a judge whether the diagnosed error is gone. They cost c_F and $c_S \ll c_F$.

Edges and patch quality. We write $e_{ij} \in \{0, 1\}$ for whether component j is a cause of category i , meaning some change to j repairs a non-trivial fraction of its failures, and $q_{ijk} \in [0, 1]$ for the repair probability of patch k when $e_{ij} = 1$. This separation matters because a negative replay outcome is ambiguous: either j is the wrong component or a_{ijk} is a bad patch. Testing several patches per cell resolves the ambiguity.

Objective. After spending at most budget B , the method accepts a set of cells $\mathcal{A}$ and, for each, a

patch k_{ij}^* . Accepting a true edge repairs a fraction q_{ijk}^* of category i ’s failures; accepting a false edge costs a regression c_{fp} , reflecting that unnecessary changes perturb behaviour on tasks that used to succeed (Vasudev et al., 2026). We measure

$$G(\mathcal{A}) = \sum_i f_i \left(1 - \prod_{j \in \mathcal{A}_i^+} (1 - q_{ijk_{ij}^*}) \right) - c_{fp} |\{(i, j) \in \mathcal{A} : e_{ij} = 0\}|, \quad (1)$$

where $\mathcal{A}_i^+ = \{j : (i, j) \in \mathcal{A}, e_{ij} = 1\}$. This is the net failure mass repaired, and report it as a fraction of the oracle value obtained by accepting exactly the true edges with their best patches.

4 CARVE

4.1 Observation model

For a trajectory labelled i and patch a_{ijk} , the probability that a full replay succeeds is

$$p_{ijk} = \begin{cases} \lambda q_{ijk} + (1 - \lambda) b & e_{ij} = 1, \\ b & e_{ij} = 0, \end{cases} \quad (2)$$

where b is the spurious recovery rate (stochastic re-generation sometimes succeeds regardless of the patch) and λ is the accuracy of the category labels used to sample trajectories; a mislabelled trajectory is treated as unaffected by the patch. A single-step check returns $z = 1$ with probability $sp + \epsilon(1 - p)$, where s and ϵ are the sensitivity and false-positive rate of the check relative to the full outcome. All four nuisance parameters can be estimated from a small calibration set on which both fidelities and a human audit are run.

4.2 Prior from LLM attributions

Let n_{ij} be the number of failures in category i that the analyst attributed to component j , and $\hat{r}$ the analyst’s attribution accuracy estimated on a small audited sample. With d the expected number of causal components per category and $\rho = d/J$ the base edge density, we set

$$\pi_{ij} = (1 - \hat{r}) \rho + \hat{r} \min(1, d n_{ij} / n_i), \quad (3)$$

so that an unreliable analyst ($\hat{r} \rightarrow 0$) yields the flat prior ρ and a reliable one concentrates mass on the components it names. Patch quality has prior $q_{ijk} \sim \text{Beta}(\alpha_q, \beta_q)$.

4.3 Posterior

Cells are independent given the data. For each cell we keep the posterior log-odds of e_{ij} and, for each patch, a posterior over q_{ijk} on a fixed grid of $G = 101$ points. With $\mathcal{D}_{ijk}$ the outcomes observed for patch k ,

$$\log \frac{P(e_{ij}=1 \mid \mathcal{D})}{P(e_{ij}=0 \mid \mathcal{D})} = \log \frac{\pi_{ij}}{1 - \pi_{ij}} + \sum_k \log \frac{\int P(\mathcal{D}_{ijk} \mid q) dP(q)}{P(\mathcal{D}_{ijk} \mid e_{ij}=0)}. \quad (4)$$

Every update multiplies one grid vector by a likelihood vector, so updates and look-aheads are vectorised dot products.

4.4 Selecting interventions

The value of the current decision for cell (i, j) , with P_{ij} its edge posterior and $\hat{Q}_{ij} = \max_k \mathbb{E}[q_{ijk} \mid \mathcal{D}]$, is

$$V_{ij} = \max(P_{ij} f_i \hat{Q}_{ij} - (1 - P_{ij}) c_{fp}, 0), \quad (5)$$

the expected contribution of the cell to Eq. 1 under the Bayes decision (accept iff the first term is positive). For an action $a = (i, j, k, \ell)$ repeated n times, let $V'_{ij}(s)$ be the value after observing s successes. The n -step knowledge gradient is

$$\text{KG}_n(a) = \sum_{s=0}^n P(s \mid \mathcal{D}, a) V'_{ij}(s) - V_{ij}, \quad (6)$$

and CARVE runs the action maximising

$$\text{score}(a) = \max_{n \in \{1, 2, 4, 8\}} \frac{\text{KG}_n(a)}{n c_\ell}. \quad (7)$$

Looking ahead several observations matters because a single binary outcome often cannot flip a decision, in which case $\text{KG}_1 = 0$ even for a valuable cell; maximising over n is the standard remedy (Frazier et al., 2008). Because cells are independent, only the observed cell’s scores change after each outcome, so selection costs $O(KG)$ per step after an initial pass over all cells.

Decisions and stopping. At any point the method accepts the cells whose current value is positive and applies their highest-posterior patch. It stops early when no action has a score above a small threshold, i.e. when no remaining experiment is expected to be worth its cost.

5 When Do Priors and Cheap Checks Help?

We analyse a single cell with fixed success probabilities p_1 (true edge, given patch) and $p_0 = b$ under a sequential rule that stops when the posterior log-odds leaves $[-A, A]$, $A = \log \frac{1-\delta}{\delta}$. Let $D_1 = \text{KL}(p_1 \parallel p_0)$ and $D_0 = \text{KL}(p_0 \parallel p_1)$ be the expected log-likelihood-ratio drift per observation under each hypothesis. Proofs, which follow Wald’s approximations (Wald, 1945), are in Appendix A.

Proposition 1 (Value of the LLM prior). *Neglecting overshoot and terminations at the wrong boundary, a cell with initial log-odds ℓ needs $N_1(\ell) = (A - \ell)/D_1$ observations in expectation if it is a true edge and $N_0(\ell) = (A + \ell)/D_0$ if not. Replacing a flat prior ℓ_ρ by LLM-derived log-odds ℓ_{ij} saves, in expectation over cells,*

$$\Delta = \frac{\rho \mathbb{E}[\delta \ell \mid e=1]}{D_1} - \frac{(1 - \rho) \mathbb{E}[\delta \ell \mid e=0]}{D_0}, \quad (8)$$

with $\delta \ell = \ell_{ij} - \ell_\rho$, observations per cell, which is positive exactly when the prior raises true edges more, relative to their base rate and drift, than it raises false ones.

Under Eq. 3, $\ell_{ij} \rightarrow \ell_\rho$ as $\hat{r} \rightarrow 0$, so $\Delta \rightarrow 0$: a useless analyst costs nothing beyond the flat prior, whereas trusting its labels outright (LLM-only) makes permanent errors. A miscalibrated analyst can make Δ negative, but only by an amount linear in the log-odds shift, after which the evidence takes over.

Proposition 2 (Cheap checks). *Let D_e^S denote the drifts of single-step outcomes, $z \sim \text{Bern}(s p_e + \epsilon(1 - p_e))$. Under the same approximations, single-step checks reach the stopping boundary at lower expected cost than full replays if and only if*

$$\frac{c_S}{c_F} < \frac{D_e^S}{D_e}, \quad (9)$$

where e is the true state of the cell.

With our default parameters ($p_1 = 0.54$, $b = 0.05$, $s = 0.85$, $\epsilon = 0.15$) the critical ratio is 0.31 for true edges and 0.44 for false ones; with a weak check ($s = 0.65$, $\epsilon = 0.35$) it falls to 0.05–0.08. Section 6.4 tests both predictions.

6 Controlled Experiments

Real agents do not reveal which component truly caused a failure, so we first evaluate in a testbed where the causal graph is known and the method’s model is deliberately misspecified.

6.1 Setup

Testbed. Each world has $I = 12$ categories with Zipfian frequencies, $J = 20$ components and $K = 3$ patches per cell. Every category has one or two true components and one *decoy* component that wrong attributions prefer, mimicking symptom-level mislabelling. Patch quality is bimodal: with probability 0.3 a patch is bad ($q \sim \text{Beta}(1, 9)$), otherwise $q \sim \text{Beta}(6, 3)$. A trajectory sampled for category i is mislabelled with probability $1 - \lambda = 0.2$, in which case it comes from another category (weighted by frequency) and is repaired only if *its own* edge to j exists. This differs from the model’s flat treatment of mislabelled traces. The analyst attributes 20 failures per category with accuracy $r = 0.5$; wrong attributions go to the decoy with probability 0.6. The analyst’s accuracy is estimated from an audit of 20 attributions. Costs are $c_F = 1$, $c_S = 0.1$, $c_{fp} = 0.02$, and $s = 0.85$, $\epsilon = 0.15$, $b = 0.05$. Full details are in Appendix B.

Baselines. *LLM-only* accepts, for every category, the most-attributed component and any component holding at least 40% of the attributions, with the first patch; this corresponds to acting on an induced taxonomy without verification, as in AdaMAST-style pipelines. *Replay-each* tests attributed hypotheses in order of attribution count with three full replays of the first patch and accepts a hypothesis if at least two succeed, mirroring do-then-verify methods such as DoVer; untested hypotheses fall back to LLM-only. *Uncertainty sampling* uses CARVE’s posterior and decision rule but replays the cell with the largest frequency-weighted uncertainty $f_i \min(P_{ij}, 1 - P_{ij})$ at full fidelity, cycling through patches. It isolates the contribution of the selection rule.

Metrics. Net gain (Eq. 1) as a fraction of the oracle, and edge F1. We report means and standard errors over 50 worlds for the main experiment and 30 for sweeps.

6.2 Main results

Figure 2 and Table 1 show that CARVE dominates every baseline at every budget. With 20 units it recovers 67% of the oracle gain, against 46% for uncertainty sampling. It passes 80% of the oracle between 50 and 75 units; uncertainty sampling needs about 200. LLM-only is stuck at 30% because it accepts decoys and bad first patches. Replay-each

Method	Budget (full-replay units)			
	20	50	100	200
LLM attribution only	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021
Replay-each (do-then-verify)	0.255 $\pm$ 0.021	0.270 $\pm$ 0.021	0.466 $\pm$ 0.027	0.468 $\pm$ 0.026
Uncertainty sampling	0.455 $\pm$ 0.016	0.581 $\pm$ 0.019	0.690 $\pm$ 0.017	0.804 $\pm$ 0.016
CARVE (full)	0.666 $\pm$ 0.019	0.787 $\pm$ 0.015	0.853 $\pm$ 0.011	0.902 $\pm$ 0.009
full replay only	0.514 $\pm$ 0.016	0.609 $\pm$ 0.020	0.697 $\pm$ 0.017	0.825 $\pm$ 0.012
myopic ($n=1$ look-ahead)	0.591 $\pm$ 0.016	0.652 $\pm$ 0.014	0.706 $\pm$ 0.016	0.729 $\pm$ 0.014
uninformed prior	0.347 $\pm$ 0.019	0.474 $\pm$ 0.017	0.631 $\pm$ 0.015	0.800 $\pm$ 0.012
one patch per cell	0.571 $\pm$ 0.019	0.636 $\pm$ 0.020	0.673 $\pm$ 0.019	0.688 $\pm$ 0.020
no label-noise term	0.663 $\pm$ 0.020	0.772 $\pm$ 0.018	0.864 $\pm$ 0.012	0.922 $\pm$ 0.006

Table 1: Net gain as a fraction of the oracle (mean $\pm$ s.e., 50 worlds). The lower block ablates CARVE.

is initially *worse* than not verifying at all: three replays of a single patch reject many true edges whose first patch is bad or whose sampled traces were mislabelled, and its fixed per-hypothesis cost exhausts small budgets on frequent but already-certain hypotheses.

Ablations. Each component of CARVE contributes (Table 1, lower block). Removing single-step checks costs 15–18 points at small budgets. Restricting the look-ahead to one observation helps early but plateaus at 73%, because many cells cannot be flipped by a single outcome and are never revisited. A flat prior costs 32 points at budget 20, and the gap closes only at large budgets. With one patch per cell, the method plateaus at 69%: it cannot distinguish a wrong component from a bad patch, so it rejects true edges for good. The label-noise term makes no measurable difference here (0.902 vs. 0.922 at budget 200, overlapping within two standard errors), which we attribute to the generator’s mislabelled traces sometimes being repairable, unlike in the model; we keep the term because it costs nothing and matters when labels are worse.

6.3 When does the LLM prior help?

Figure 3 varies the analyst’s accuracy r . As Proposition 1 predicts, CARVE improves monotonically with r and never falls below the flat-prior variant: at $r = 0.1$ it scores 0.51 ± 0.02 against 0.48 ± 0.03 , because the calibrated prior collapses to the flat one. Acting on attributions directly is harmful below $r \approx 0.35$, where LLM-only has a *negative* net gain (regressions outweigh repairs), and even at $r = 0.95$ it reaches only 71% of the oracle against 96% for CARVE, since it cannot detect bad patches. Replay-each improves with r but stays below the flat-prior variant of CARVE throughout.

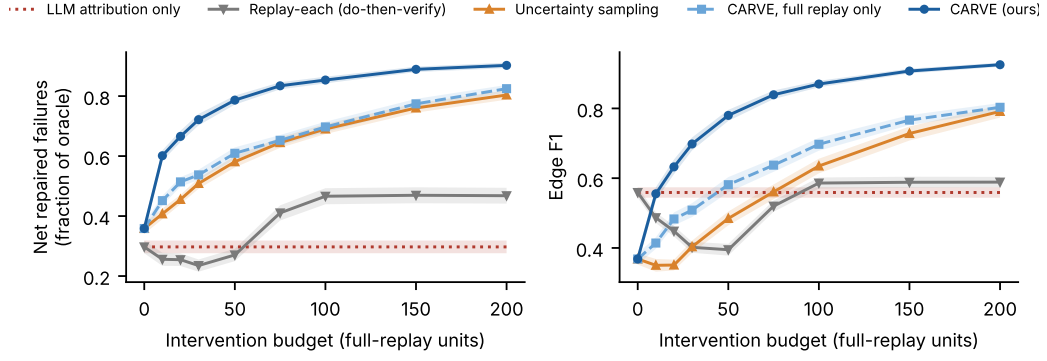


Figure 2: Net repaired failure mass (left) and edge F1 (right) against intervention budget, 50 worlds, shaded ± 1 s.e.

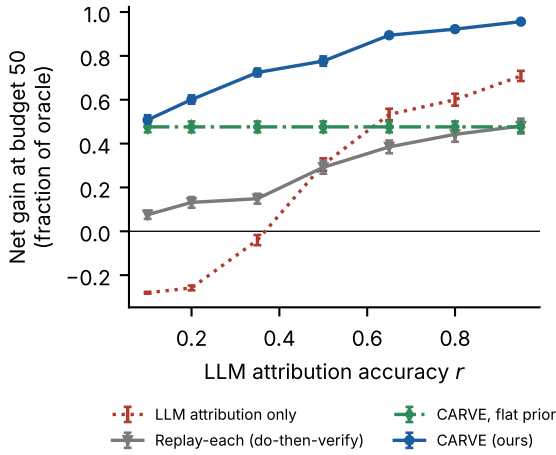


Figure 3: Net gain at budget 50 as the analyst’s attribution accuracy r varies (30 worlds, ± 1 s.e.).

Model assumption (true value)	Net gain
All nuisance parameters as in the generator	0.776 ± 0.021
Spurious rate b : 0.01 (0.05)	0.811 ± 0.012
Spurious rate b : 0.15 (0.05)	0.826 ± 0.016
Label accuracy λ : 1.0 (0.8)	0.768 ± 0.022
Label accuracy λ : 0.6 (0.8)	0.794 ± 0.017
Check s/ϵ : .95/.05 (.85/.15)	0.717 ± 0.016
Check s/ϵ : .70/.30 (.85/.15)	0.636 ± 0.021
Patch prior Beta(1, 1) (Beta(2, 2))	0.811 ± 0.015
Patch prior Beta(5, 2) (Beta(2, 2))	0.777 ± 0.020

Table 2: CARVE with misspecified nuisance parameters, budget 50, 30 worlds.

6.4 When do cheap checks help?

Figure 4 matches Proposition 2. With the default check, CARVE beats the full-replay-only variant for cost ratios up to 0.35 (0.64 ± 0.02 vs. 0.61 ± 0.03) and ties at 0.5 (0.59 vs. 0.61), consistent with the predicted crossover between 0.31 and 0.44. At cost 0.1, a check of quality $s - \epsilon = 0.5$ (predicted crossover 0.14–0.22) still helps, while one of quality 0.3 (predicted 0.05–0.08) brings no gain; CARVE then simply stops choosing it, so a useless check does not hurt.

6.5 Robustness and scale

Table 2 perturbs the model’s nuisance parameters. Most errors change the result by less than three standard errors, and several “wrong” settings do slightly better than the generator’s values, a reminder that the model is misspecified in other ways too. The one setting that matters is the quality of

the single-step check: overstating it (0.717) makes the method trust cheap checks too much, and understating it (0.636) makes it under-use them. This parameter should be calibrated on a small set where both fidelities are run.

On a larger graph ($I = 20$, $J = 50$; Table 3 in Appendix C) the gap widens: at budget 50 CARVE reaches 78% of the oracle against 61% for its full-replay variant and 50% for uncertainty sampling.

6.6 Inside a self-improvement loop

Finally we embed each method in a loop. The agent starts at 40% success. In each round the analyst attributes 240 new failures, sampled in proportion to the current failure distribution; the method spends 40 units; accepted patches are applied, which repairs failures (true edges) or adds a regression of c_{fp} times the current failure mass (false edges); and the failure distribution is recomputed, so priorities shift towards categories that remain. Posteriors carry over between rounds.

Figure 5 shows that CARVE reaches 80% success after one round and 85% after two, close to its plateau of 86% (the oracle ceiling is 91.5%). Uncertainty sampling reaches 84.5% only after six rounds, i.e. with three times the cumulative budget,

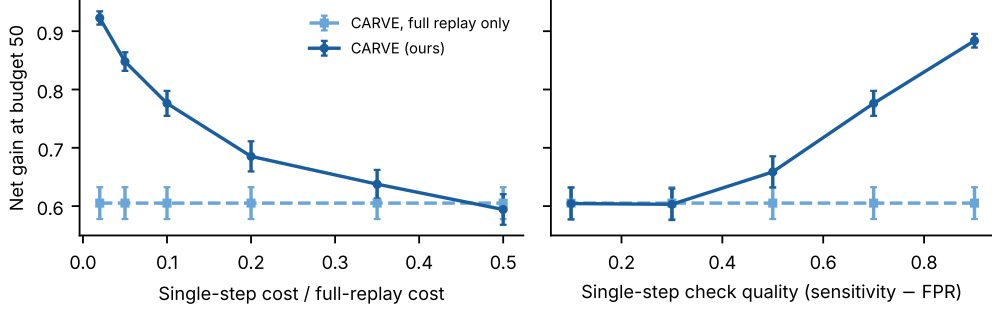


Figure 4: Effect of the single-step check’s cost (left, quality fixed at $s=0.85$, $\epsilon=0.15$) and quality (right, cost fixed at 0.1) on net gain at budget 50.

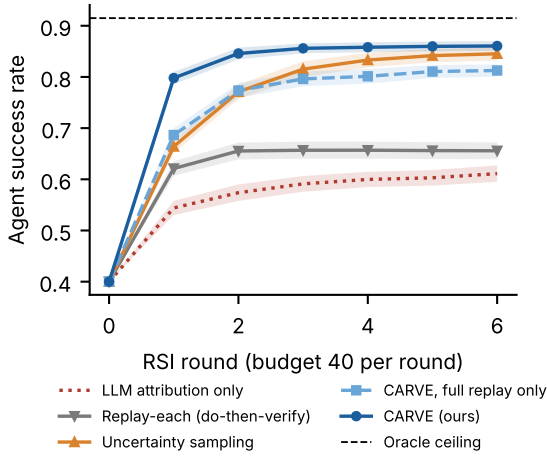


Figure 5: Agent success over six rounds of attribute-verify-patch, with a budget of 40 units per round (30 worlds, ± 1 s.e.).

and the full-replay variant of CARVE plateaus at 81%. LLM-only and Replay-each plateau at 61% and 66%: once their accepted false edges and rejected true edges are fixed in place, new attributions do not correct them.

7 Real-Agent Study

Environment. ShopDesk is a customer-service environment with a seeded database of customers, orders, products and policies, six deterministic tools and exactly checkable answers (refund amounts, order counts, spend totals). Determinism makes replay from any step exact: recorded tool results are re-used and only the agent’s generation is re-sampled. The agent is a ReAct-style tool-use loop (Yao et al., 2023) assembled from 11 named components (four system-prompt sections, six tool descriptions and the step limit), and every

step records which components shaped it.

Ground truth by fault injection. Following the fault-injection idea of Zhang et al. (2026), each training task runs under exactly one of five injected faults or none: a tool description that mis-states price units, a list tool whose description omits pagination, a format instruction that rounds to whole dollars, a planning instruction that hard-codes the wrong refund policy, and a step limit that is too small. The responsible component of every failure is therefore known. After the analyst attributes the failures, induces a taxonomy in TnT-LLM style (Wan et al., 2024) and labels each failure, a category–component edge is defined as true when at least 30% of the category’s failures come from that component’s fault.

Protocol. Every method receives the same attributions, taxonomy and token budget. Full replays resume from the attributed step; single-step checks regenerate that step and use an LLM judge (Zheng et al., 2023). Patches are generated lazily by the analyst, three per cell, and their tokens are charged to the method. Cost units are calibrated from measured tokens. We report edge precision and recall, and the held-out success rate after applying each method’s accepted patches.

Status. [TODO: The real-agent experiments have not yet been run: the environment used to prepare this draft had no model API access. The pipeline is implemented and tested end to end with a mock model (agent_exp/); results, the measured cost ratio c_S/c_F , the analyst’s measured attribution accuracy and a qualitative analysis of accepted patches will be added here. No numbers in this section should be read as results.]

8 Discussion

What the analyst LLM is for. Our results suggest a change of role for LLM attribution in self-improvement loops: from judge to prior. When attributions are good, they focus the budget; when they are bad, a calibrated prior falls back to an uninformed one, whereas acting on them directly causes regressions that later rounds do not undo.

Why patches must be modelled. The single largest failure mode of fixed-replay verification in our experiments is rejecting a true edge because its first patch was bad. Treating patch quality as a separate latent variable, and generating more than one patch, is cheap and removes this failure mode.

Beyond categories and components. The graph formulation does not depend on how categories or components are defined. Categories could come from a fixed taxonomy such as MAST, from AdaMAST, or from clustering; components could be finer (individual sentences of a prompt) or coarser (whole agents in a multi-agent system). The replay primitive can be any of the intervention methods in §2.

9 Conclusion

We framed the verification step of failure-driven agent self-improvement as budgeted learning of a causal category–component graph, and proposed CARVE, which treats LLM attributions as a calibrated prior, separates causes from patch quality, and allocates replays across cells, patches and fidelities by expected value per unit cost. In controlled experiments it reaches the same repaired failure mass as the strongest baseline with about a third of the budget and degrades gracefully when attributions are poor. The real-agent evaluation is the main open item.

Limitations

The quantitative results in this draft come from a simulated testbed. Although the generator is deliberately richer than the model (systematic decoy attributions, bimodal patch quality, mislabelled traces that can still be repaired), real agents may violate the model in ways we did not simulate, most notably through interactions between components, where two changes are needed together, and through patches that help one category while hurting another beyond a constant regression cost. Our cell-independence assumption ignores both. The

method also assumes that failures can be replayed from the attributed step; this holds for deterministic tools and sandboxes but not for live web environments. The single-step check relies on an LLM judge whose quality must be calibrated, and Table 2 shows that this calibration matters. Finally, the propositions use Wald’s approximations and ignore overshoot and errors at the boundary; they are guides to when the method helps, not guarantees.

Ethics Statement

Automated self-improvement changes an agent’s behaviour without a human reviewing each change. CARVE makes these changes more reliable by requiring interventional evidence before acceptance, but accepted patches should still be reviewed before deployment in settings where errors carry real costs. Our environment uses synthetic data only.

References

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnab Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2025. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*.
- Akash Bonagiri, Devang Borkar, Gerard Janno Andersonias, Setareh Rafatirad, and Houman Homayoun. 2026. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*.
- Mert Cemri, Andrei Cojocaru, Melissa Pan, Shu Liu, Shubham Agarwal, Alexander Krentsel, Jay Tang, Kannan Ramchandran, Joseph E. Gonzalez, Matei Zaharia, Alexandros G. Dimakis, and Ion Stoica. 2026. Fantastic adaptive taxonomies and how to use them. *arXiv preprint arXiv:2607.16387*.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.
- Kathryn Chaloner and Isabella Verdinelli. 1995. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304.
- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*.

619	Peter I. Frazier, Warren B. Powell, and Savas Dayanik.	learning. In <i>Advances in Neural Information Processing Systems</i> .	673
620	2008. A knowledge-gradient policy for sequential		674
621	information collection. <i>SIAM Journal on Control</i>		
622	and <i>Optimization</i> , 47(5):2410–2439.		
623	Alain Hauser and Peter Bühlmann. 2014. Two opti-	Rakshith Vasudev, Melisa Russak, Dan Bikel, and	675
624	mal strategies for active learning of causal models	Waseem Alshikh. 2026. Accurate failure prediction	676
625	from interventional data. <i>International Journal of</i>	in agents does not imply effective failure prevention.	677
626	<i>Approximate Reasoning</i> , 55(4):926–939.	<i>arXiv preprint arXiv:2602.03338</i> .	678
627	Shengran Hu, Cong Lu, and Jeff Clune. 2025. Auto-	Abraham Wald. 1945. Sequential tests of statistical	679
628	ated design of agentic systems. In <i>International</i>	hypotheses. <i>The Annals of Mathematical Statistics</i> ,	680
629	<i>Conference on Learning Representations</i> .	16(2):117–186.	681
630	Kirthevasan Kandasamy, Gautam Dasarathy, Jeff	Mengting Wan, Tara Safavi, Sujay Kumar Jauhar, Yujin	682
631	Schneider, and Barnabás Póczos. 2016. The multi-	Kim, Scott Counts, Jennifer Neville, Siddharth Suri,	683
632	fidelity multi-armed bandit. In <i>Advances in Neural</i>	Chirag Shah, Ryen W. White, Longqi Yang, Reid	684
633	<i>Information Processing Systems</i> .	Andersen, Georg Buscher, Dhruv Joshi, and Nagu	685
634	Jiazheng Li, Emine Yilmaz, Bei Chen, and Dieu-	Rangan. 2024. TnT-LLM: Text mining at scale with	686
635	Thu Le. 2026. Towards self-improving error di-	large language models. In <i>Proceedings of the 30th</i>	687
636	agnosis in multi-agent systems. <i>arXiv preprint</i>	<i>ACM SIGKDD Conference on Knowledge Discovery</i>	688
637	<i>arXiv:2604.17658</i> .	and <i>Data Mining</i> .	689
638	Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok,	Xiaojun Wu, Cehao Yang, Honghao Liu, Xueyuan Lin,	690
639	Daniel Guo, Sahil Arun Nale, Charles Fleming,	Wenjie Zhang, Zhichao Shi, Xuhui Jiang, Chengjin	691
640	and Guang Cheng. 2026. REFLECT: Intervention-	Xu, Jia Li, and Jian Guo. 2026. Bayesian-Agent:	692
641	supported error attribution for silent failures in LLM	Posterior-guided skill evolution across LLM agent	693
642	agent traces. <i>arXiv preprint arXiv:2606.09071</i> .	harnesses. <i>arXiv preprint arXiv:2606.08348</i> .	694
643	Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tian-	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak	695
644	wei Yue, Chi Wang, Jian Kang, Qingyun Wu, and	Shafran, Karthik Narasimhan, and Yuan Cao. 2023.	696
645	Huazheng Wang. 2026. Who&when pro: Can LLMs	ReAct: Synergizing reasoning and acting in language	697
646	really attribute failures in AI agents? <i>arXiv preprint</i>	models. In <i>International Conference on Learning</i>	698
647	<i>arXiv:2607.09996</i> .	<i>Representations</i> .	699
648	Andrea Locatelli, Maurilio Gutzeit, and Alexandra Car-	Guibin Zhang, Junhao Wang, Junjie Chen, Wangchun-	700
649	pentier. 2016. An optimal algorithm for the thresh-	shu Zhou, Kun Wang, and Shuicheng Yan. 2026.	701
650	olding bandit problem. In <i>Proceedings of the 33rd</i>	AgentTracer: Who is inducing failure in the LLM	702
651	<i>International Conference on Machine Learning</i> .	agentic systems? In <i>International Conference on</i>	703
652	Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qing-	<i>Learning Representations</i> . ArXiv:2509.03312.	704
653	wei Lin, Saravan Rajmohan, and Dongmei Zhang.	Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange,	705
654	2025. DoVer: Intervention-driven auto debug-	and Jeff Clune. 2025a. Darwin Gödel machine:	706
655	ging for LLM multi-agent systems. <i>arXiv preprint</i>	Open-ended evolution of self-improving agents.	707
656	<i>arXiv:2512.06749</i> .	<i>arXiv preprint arXiv:2505.22954</i> .	708
657	Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler	Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu,	709
658	Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon,	Zhiguang Han, Jingyang Zhang, Beibin Li, Chi	710
659	Nouha Dziri, Shrimai Prabhunoye, Yiming Yang,	Wang, Huazheng Wang, Yiran Chen, and Qingyun	711
660	Shashank Gupta, Bodhisattwa Prasad Majumder,	Wu. 2025b. Which agent causes task failures	712
661	Katherine Hermann, Sean Welleck, Amir Yazdan-	and when? on automated failure attribution of	713
662	bakhsh, and Peter Clark. 2023. Self-refine: Iterative	LLM multi-agent systems. In <i>Proceedings of the</i>	714
663	refinement with self-feedback. In <i>Advances in Neu-</i>	<i>42nd International Conference on Machine Learning</i> .	715
664	<i>ral Information Processing Systems</i> .	ArXiv:2505.00212.	716
665	Judea Pearl. 2009. <i>Causality: Models, Reasoning, and</i>	Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu	717
666	<i>Inference</i> , 2nd edition. Cambridge University Press.	Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL:	718
667	Jaineet Shah. 2026. Causal agent replay: Counterfactual	LLM agents are experiential learners. In <i>Proceedings</i>	719
668	attribution for LLM-agent failures. <i>arXiv preprint</i>	<i>of the AAAI Conference on Artificial Intelligence</i> .	720
669	<i>arXiv:2606.08275</i> .	Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan	721
670	Noah Shinn, Federico Cassano, Ashwin Gopinath,	Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin,	722
671	Karthik Narasimhan, and Shunyu Yao. 2023. Re-	Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang,	723
672	flexion: Language agents with verbal reinforcement	Joseph E. Gonzalez, and Ion Stoica. 2023. Judging	724
		LLM-as-a-judge with MT-bench and chatbot arena.	725
		In <i>Advances in Neural Information Processing Sys-</i>	726
		<i>tems, Datasets and Benchmarks Track</i> .	727

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. 2026. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*.

A Proofs

Proposition 1. Let $L_t = \ell + \sum_{u \leq t} \log \frac{P(o_u|e=1)}{P(o_u|e=0)}$ be the posterior log-odds after t observations. Under $e = 1$, the increments are i.i.d. with mean $D_1 = \text{KL}(p_1 \| p_0) > 0$. The rule stops at $N = \inf\{t : L_t \notin (-A, A)\}$. By Wald’s identity, $\mathbb{E}_1[L_N - \ell] = D_1 \mathbb{E}_1[N]$. Neglecting overshoot and the probability (at most of order δ) of exiting at $-A$, $L_N = A$, hence $\mathbb{E}_1[N] = (A - \ell)/D_1$. Symmetrically, under $e = 0$ the drift is $-D_0$ and $\mathbb{E}_0[N] = (A + \ell)/D_0$. Taking the difference between ℓ_ρ and ℓ_{ij} and averaging over cells with $P(e=1) = \rho$ gives Δ . When the patch quality q is unknown, p_1 is replaced by its posterior predictive, which changes D_1 but not the form of the result.

Proposition 2. By the same argument a single-step test needs $(A \mp \ell)/D_e^S$ observations at cost c_S each, and a full-replay test needs $(A \mp \ell)/D_e$ at cost c_F . The single-step test is cheaper iff $c_S/D_e^S < c_F/D_e$. Since the single-step outcome is a garbling of the full outcome, the data-processing inequality gives $D_e^S \leq D_e$, so the critical ratio is at most one.

B Simulation Details

Category frequencies are $f_i \propto 1/i$. For each category the number of true components is uniform on $\{1, 2\}$ and the decoy is a further distinct component. The audit estimate $\hat{r}$ is a binomial proportion from 20 audited attributions. The prior uses $d = 1.5$. The model’s grid has 101 points; look-ahead horizons are $\{1, 2, 4, 8\}$; the stopping threshold is 10^{-6} . Replay-each uses three replays and accepts with at least two successes. Uncertainty sampling switches to an untried patch once the current patch has three full observations. In the loop experiment, 240 attributions per round are allocated to categories by a multinomial draw with

the current failure distribution. All randomness is seeded; the code reproduces every number in this paper with `python -m sim.experiments all`.

C Additional Results

Method	50	100	300
LLM attribution only	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033
Replay-each (do-then-verify)	0.224 $\pm$ 0.034	0.400 $\pm$ 0.037	0.531 $\pm$ 0.032
Uncertainty sampling	0.502 $\pm$ 0.022	0.637 $\pm$ 0.020	0.822 $\pm$ 0.015
CARVE, full replay only	0.613 $\pm$ 0.013	0.710 $\pm$ 0.012	0.834 $\pm$ 0.010
CARVE (ours)	0.780 $\pm$ 0.016	0.860 $\pm$ 0.011	0.933 $\pm$ 0.008

Table 3: Net gain on a larger graph ($I=20$, $J=50$), 20 worlds.